Rust - HashMap

1. Caractéristique d'une HashMap

Les `HashMap` de Rust sont conceptuellement très proches des dictionnaires Python (`dict`) et des structures similaires dans d'autres langages. Cependant Rust demande d'être plus explicite sur la mutabilité, la possession, et la gestion des cas "clé absente".

Une HashMap en Rust est une implémentation technique avancée qui offre une sécurité contre les attaques de déni de service basée sur les collisions (DoS) grâce à l'algorithme utilisé (SpiHash).

- Type : `HashMap<K, V>`
- Stocke des paires clé → valeur
- Accès par clé (de n'importe quel type), pas par indice
- Toutes les clés ont le même type `K`, toutes les valeurs le même type `V` (homogène)
- Les données sont stockées sur le tas (heap)

2. Import et création

```rust
use std::collections::HashMap; // ⚠️ Ne pas oublier

// Création vide (type inféré ou annoté)
let mut book = HashMap::new();
let mut book: HashMap<String, String> = HashMap::new();
let mut book = HashMap::<String, String>::new();

// Insertion
book.insert(String::from("author"), "Kerouac");
```

Il est également possible d'initialiser une HashMap à partir d'un tableau :

```rust
let distance_solaire = HashMap::from([
    ("Mercure", 0.4),
    ("Vénus", 0.7),
    ("Terre", 1.0),
    ("Mars", 1.5),
]);
```

3. Règles de possession (*ownership*)

Type	Comportement
`Copy` (`i32`, `bool`, `f64`, etc.)	Copie dans la map → la variable originale reste valide
Non `Copy` (`String`, `Vec`, etc.)	Déplacement (*move*) → la variable n'est plus utilisable

```
let cle = String::from("author");   // String = non-Copy
book.insert(cle, "Kerouac");
// cle n'est plus valide ici ❌
```

4. Accès aux valeurs

Avec `.get()`

```
let score = scores.get(&team_name).copied().unwrap_or(0);
// Retourne un Option<&V>
// .copied() transforme Option<&i32> en Option<i32>
// .unwrap_or(0) donne une valeur par défaut
```

Itération

```
for (key, value) in &scores {
    println!("{key}: {value}");
}
// L'ordre est arbitraire (non garanti)
```

Les types String

Lorsque les collections stockent des valeurs possédées (`String`), on peut toujours les interroger en utilisant des références (`&str`).

```
use std::collections::HashMap;

fn main() {
    let mut book = HashMap::<String, String>::new();
    book.insert("Kerouac".to_string(), "Sur la Route".to_string());
```

```
    book.insert("Djian".to_string(), "Sotos".to_string());
    book.insert("Hemingway".to_string(), "Paris est une
fête".to_string());
    let authors_to_find = ["Joyce", "Kerouac", "Hemingway", "Nabokov"];
    for &author in &authors_to_find {
        match book.get(author) {
            Some(title) => println!("{author} : {title}"),
            None => println!("Pas de livre référencé pour {author}"),
        }
    }
}
```

Pattern matching **&&**

Le tableau `authors_to_find` contient des valeurs de type `&str` (= références vers des chaînes statiques). Le tableau est de type `[&str; 4]`.
Dans la boucle `for`, `&authors_to_find` emprunte le tableau (sans le consommer).
Avec `for author in &authors_to_find`, chaque `author` est alors de type `&&str`, c'est-à-dire une référence vers une référence, soit une référence vers un `&str`
`&author` = *pattern matching*. Cela ne crée pas une référence mais déconstruit une référence reçue. `author` reçoit `&&str`, mais le motif `&author` "retire" un niveau de `&`, ainsi `author` est de type `&str`. Et voilà notre `&str` que peut recevoir la méthode `.get()`. `&&`, pour capturer la référence d'une référence, n'est pas `&` + `&` (Voir ici).
Voir également : https://medium.com/@robertgrosse/ref-patterns-destructuring-and-invisible-borrows-2f8ae6902656

Rechercher une valeur par une clé

```
println!("{}", book["Djian"]); // Sotos
println!("{}", book["Joyce"]); // Panic ! car clé absente
```

5. Mise à jour d'une entrée

Action	Code	Comportement
Écraser	`map.insert(clé, nouvelle_valeur)`	Remplace l'ancienne valeur

Action	Code	Comportement
Insérer si absent	`map.entry(clé).or_insert(valeur)`	Ne modifie rien si la clé existe déjà
Modifier la valeur existante	`*compteur += 1`	Utilise la référence mutable retournée par `or_insert`

6. L'API `Entry` (très importante)

L'API `Entry` permet l'emploi de méthodes complexes pour obtenir, définir, mettre à jour et supprimer des clés et
leurs valeurs.

```
let compteur = map.entry(mot).or_insert(0);
*compteur += 1;
```

- `entry(clé)` retourne un enum `Entry` (`Occupied` ou `Vacant`)
- `.or_insert(v)` insère `v` si la clé n'existe pas, retourne une référence mutable `&mut V`
- Permet de faire une seule recherche pour "lire ou écrire"

```
use std::collections::HashMap;

fn joyce_title() -> String {
    "Ulysse".to_string()
}

fn main() {
    let mut book = HashMap::<String, String>::new();
    book.insert("Kerouac".to_string(), "Sur la Route".to_string());
    book.insert("Djian".to_string(), "Sotos".to_string());
    book.insert("Hemingway".to_string(), "Paris est une
fête".to_string());
    // Insérer une paire clé/valeur à l'aide d'une fonction :
    book.entry("Joyce".to_string()).or_insert_with(joyce_title);
    // Modifier une entrée avant une insertion avec une mutation sur
place :
    book.entry("Hemingway".to_string())
        .and_modify(|hemingway| *hemingway = "L'été
```

```
Dangereux".to_string())
        .or_insert("Paradis perdu".to_string());
    println!("{book:?}"); // {"Djian": "Sotos", "Joyce": "Ulysse",
"Kerouac": "Sur la Route", "Hemingway": "L'été Dangereux"}
}
```

7. Exemple classique : compteur de mots

```
let texte = "bonjour monde merveilleux monde";
let mut map = HashMap::new();
for mot in texte.split_whitespace() {
    let compteur = map.entry(mot).or_insert(0);
    *compteur += 1;
}
println!("{map:?}"); // {"monde": 2, "bonjour": 1, "merveilleux": 1}
```

8. Attention : emprunts multiples (`E0499`)

```
let ref1 = map.entry("a").or_insert(0);   // 1er emprunt mutable
let ref2 = map.entry("b").or_insert(0);   // ❌ 2e emprunt mutable →
erreur !
```

Solution : utiliser des blocs pour limiter la durée de vie des [références]

```
{
    let ref1 = map.entry("a").or_insert(0);
    *ref1 += 1;
} // ref1 libéré ici
let ref2 = map.entry("b").or_insert(0);   // ✅ ok
```

9. Utilisation avec des types de clés personnalisés

La façon la plus simple d'utiliser HashMap avec un type de clé personnalisé est de dériver `Eq` et `Hash`. Nous devons également dériver `PartialEq`.

```
use std::collections::HashMap;

#[derive(Hash, Eq, PartialEq, Debug)]
struct Book {
    author: String,
```

```rust
    title: String,
}

impl Book {
    /// Crée un nouveau livre.
    fn new(name: &str, book: &str) -> Book {
        Book {
            author: name.to_string(),
            title: book.to_string(),
        }
    }
}

fn main() {
    // Utiliser une HashMap pour stocker les livres.
    let all_books = HashMap::from([
        (Book::new("Hemingway", "Paris est une fête"), 1964),
        (Book::new("Djian", "Sotos"), 1993),
        (Book::new("Kerouac", "Sur la Route"), 1957),
    ]);

    // Utiliser l'implémentation dérivée pour afficher l'état des
livres.
    for (book, year) in &all_books {
        println!("{book:?} publié en {year}");
    }
}
```

Sortie :

```
Book { author: "Hemingway", title: "Paris est une fête" } publié en
1964
Book { author: "Djian", title: "Sotos" } publié en 1993
Book { author: "Kerouac", title: "Sur la Route" } publié en 1957
```

10. Quand utiliser une `HashMap` ?

Pour :

- Associations clé → valeur (ex: scores, configurations)

- Compter des occurrences
  Pattern idiomatique en Rust pour compter des fréquences :

```
for element in collection {
    let compteur = map.entry(element).or_insert(0);
    *compteur += 1;
}
```

- Indexer des données par un identifiant

Préférer un vecteur (`Vec`) pour :

- Accès par indice numérique
- Ordre d'insertion important

11. Méthodes avec les HashMap

Nous avons déjà vu les méthodes `new()` , `insert(k, v)` , `get(&k)` , `entry(k)` , voyons d'autres méthodes couramment utilisées :

Méthode	Utilisation	Explication
`get_mut(&k)`	`map.get_mut(&key_name)`	Retourne `Option<&mut V>` (référence mutable). Permet de modifier la valeur directement.
`remove(&k)`	`map.remove(&key_name)`	Supprime une entrée et retourne `Option<V>` (la valeur supprimée, ou `None` si inexistante).
`contains_key(&k)`	`map.contains_key(&clé)`	Retourne `true` si la clé existe dans la HashMap.
`len()`	`map.len()`	Retourne le nombre d'entrées (paires clé-valeur) dans la HashMap.
`is_empty()`	`map.is_empty()`	Retourne `true` si la HashMap ne contient aucune entrée.
`clear()`	`map.clear();`	Supprime toutes les entrées de la HashMap.

Méthode	Utilisation	Explication
`keys()`	`for k in map.keys()`	Itère sur toutes les clés (ordre arbitraire).
`values()`	`for v in map.values()`	Itère sur toutes les valeurs (ordre arbitraire).
`values_mut()`	`for v in map.values_mut()`	Itère sur des références mutables vers les valeurs.
`clone()`	`let nouvelle = map.clone();`	Copie en profondeur (deep copy) de toute la HashMap. Les clés/valeurs doivent implémenter `Clone`.
`from()`	`HashMap::from([(k1,v1), (k2,v2)])`	Crée une HashMap à partir d'un tableau de paires

Méthodes spécifiques :

Méthode	Utilisation	Explication
`reserve(n)`	`map.reserve(10);`	Réserve de la capacité pour au moins `n` entrées supplémentaires (optimisation).
`shrink_to_fit()`	`map.shrink_to_fit();`	Réduit la capacité de la HashMap au minimum nécessaire.
`drain()`	`map.drain().collect::<Vec<_>>()`	Supprime toutes les entrées et les retourne sous forme d'itérateur.
`retain(f)`	`map.retain(|k, v| *v > 0);`	Garde seulement les entrées qui satisfont la condition.

12. Ressources

- Documentation officielle : std::collections::HashMap
- **Rust Book** : Chapitre 8 : Storing Keys with Associated Values in Hash Maps]